

SOAP (SOAP Web Services Execution Engine)

Milenko Burlica

28 July, 2026

Table of Contents

1	 Automatische HTTP-Client-Konfiguration & Ausfallsicherheit.....	4
2	 Engine-spezifische Parameter.....	5
3	 Ordnerstruktur & Service-Mapping (Das Metadaten-Prinzip)	6
3.1	1. Verzeichnis-Struktur.....	6
3.2	2. Die Kern-Dateien eines Kommandos	6
3.3	3. Der Lebenszyklus der Request-Generierung (Runtime-Auflösung).....	7
4	 Unterstützte Operationen (op).....	8
4.1	1. Operation: EXEC (SOAP-Call ausführen)	8
4.2	2. Operation: ASSERT (Validierung via XPath)	8
4.3	3. Operation: BUFFER (Datenextraktion via XPath).....	9
5	 Konsole-Logging & Live-Ausgabe (Log Alignment).....	10
6	 Intelligentes XML-Tag-Handling: {NULL} und {EMPTY}	11

Die **SOAP (SOAP Web Services Execution Engine)** dient zur automatisierten Ausführung von XML-basierten Web-Service-Anfragen (SOAP-Calls) sowie zur anschließenden Validierung und Datenextraktion aus den erhaltenen API-Antworten. Sie reagiert dediziert auf den Testschritt-Typ `type: SOAP`.

 **Das Entkopplungs-Prinzip (Separation of Concerns)** Die SOAP-Engine trennt die HTTP-Kommunikation strikt von der Datenanalyse:

- Ein SOAP-Aufruf wird einmalig abgesetzt (`op: EXEC`) und das gesamte Antwort-Objekt (Statuscode, XML-Body, Header) unter einem eindeutigen Namen (`response`) im Execution-Context zwischengespeichert.
- Anschließend können beliebig viele Prüfungen (`op: ASSERT`) oder Variablen-Extraktionen (`op: BUFFER`) auf diesem gespeicherten Ergebnis durchgeführt werden, ohne den Ziel-Server durch erneute Aufrufe zu belasten.

1 Automatische HTTP-Client-Konfiguration & Ausfallsicherheit

- **SSL Trust-All (Unsafe Client):** Der interne `HttpClient` ist standardmäßig so konfiguriert, dass er alle SSL/TLS-Zertifikate (selbstsignierte Zertifikate im internen Netzwerk) ohne Validierung akzeptiert (`TrustManager` umgeht X509-Prüfungen).
- **Timeout-Schutz:** Während das Verbindungs-Timeout des Clients hart auf **10 Sekunden** gesetzt ist, besitzt der eigentliche SOAP-Request ein großzügiges Lese-Timeout von **15 Sekunden** (`Duration.ofSeconds(15)`), um auch auf langsame Backend-Systeme stabil zu reagieren.
- **MTOM & GZIP Support:** Die Engine dekomprimiert GZIP-Antworten automatisch im Hintergrund. Zudem besitzt sie einen Bereinigungs-Mechanismus, der MTOM/Multipart-Envelopes filtert und nur den reinen `<soap:Envelope>` extrahiert.

2 Engine-spezifische Parameter

Neben den Standardfeldern besitzt die SOAP-Engine spezielle Steuerungs-Parameter im YAML:

- **endpoint** (String): Die Basis-URL des SOAP-Zielservers. Kann hartkodiert oder dynamisch über Umgebungs-Variablen geladen werden (z. B. `http://{E[users.serverip]}`).
- **command** (String): Die spezifische SOAP-Route bzw. der vordefinierte Service-Pfad (z. B. `soap.ginosimulator.remove_card`).
- **response** (String, Pflichtfeld): Der eindeutige Schlüssel, unter dem das Antwort-Objekt im Speicher abgelegt (bei `EXEC`) oder referenziert wird (bei `ASSERT` / `BUFFER`).
- **source** (String): Bestimmt den Zielbereich für Prüfungen oder Extraktionen. Erlaubte Werte: `STATUS` , `BODY` oder `HEADERS` .
- **path** (String): Die genaue Position der Daten. Bei `BODY` wird ein standardisierter **XPath-Ausdruck** (z. B. `/Envelope/Body/RemoveCardResponse/status`) verwendet. Bei `HEADERS` wird der exakte Header-Name eingetragen.

3 Ordnerstruktur & Service-Mapping (Das Metadaten-Prinzip)

Die SOAP-Engine nutzt das Prinzip "**Convention over Configuration**". Hinter jedem logischen Namen im YAML-Feld `command` verbirgt sich eine feste Verzeichnisstruktur im Framework, die alle technischen Details des XML-Requests kapselt. Ohne diese darunterliegenden Metadaten und XML-Templates wäre die Engine nicht in der Lage, einen gültigen SOAP-Request abzusetzen.

3.1 1. Verzeichnis-Struktur

Der Punkt (`.`) im `command` -Namen wird vom Framework als Pfad-Trennzeichen (`/`) interpretiert. Für das Kommando `soap.ginosimulator.remove_card` sucht das Framework im Ressourcen-Ordner nach folgendem Verzeichnis:

Plaintext

```
src/test/resources/
├─ soap/
│   └─ ginosimulator/
│       └─ remove_card/
│           ├── metadata.json      <-- Technische HTTP-Konfiguration (Methode,
Header, URL)
│           └─ request.xml        <-- XML-Payload-Template (SOAP Envelope)
```

3.2 2. Die Kern-Dateien eines Kommandos

- `metadata.json` (**Technische Definition**): Diese Datei definiert die HTTP-Methode (für SOAP standardmäßig `POST`), die relative URL (inklusive des dynamischen Endpunkt-Platzhalters) und die erforderlichen SOAP-Header (z.B. `SOAPAction`).

JSON

```
{
  "url" : "{{endpoint}}/simulator/remove",
  "method" : "POST",
  "headers" : {
    "Content-Type" : "text/xml;charset=UTF-8",
    "SOAPAction" : ""
  }
}
```

Der Platzhalter `{{endpoint}}` wird zur Laufzeit automatisch durch den aktuellen Wert des Feldes `endpoint` aus dem YAML-Testschritt ersetzt.

- `request.xml` (**Payload-Template**): Diese Datei enthält die XML-Struktur des SOAP-Envelopes, der an den Server gesendet wird. Hier werden Platzhalter definiert, die entweder globale

Umgebungsvariablen (`E[...]`) oder dynamische Werte aus dem Laufzeit-Buffer (`B[...]`) referenzieren.

XML

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Header/>
  <soapenv:Body>
    <RemoveRequest>
      <nummer>{E[users.nummer]}</nummer>
      <slot>{B[slot]}</slot>
    </RemoveRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

3.3 3. Der Lebenszyklus der Request-Generierung (Runtime-Auflösung)

Wenn die Engine `op: EXEC` ausführt, läuft im Hintergrund folgender Prozess ab:

1. **Laden:** Die Engine öffnet den Zielordner basierend auf dem `command` -Namen und liest `metadata.json` und `request.xml` ein.
2. **Injektion:** Der Platzhalter `{{endpoint}}` in der URL wird durch die im YAML-Schritt definierte `endpoint` -URL ersetzt.
3. **Parsing & XML-Bereinigung:** Die Engine scannt die `request.xml` . `{E[...]}` und `{B[...]}` -Platzhalter werden aufgelöst. Danach läuft die XML-Struktur durch den **DOM-Null-Node-Filter** (siehe unten), um optionale XML-Tags dynamisch zu verarbeiten.

4 Unterstützte Operationen (op)

4.1 1. Operation: EXEC (SOAP-Call ausführen)

Erstellt und sendet einen HTTP-POST-Request basierend auf den konfigurierten Parametern, der geladenen `metadata.json` und dem aufgelösten, bereinigten `request.xml`.

YAML

```
- type: SOAP
  op: EXEC
  command: soap.ginosimulator.remove
  endpoint: http://{E[users.serverip]}
  response: response_soap1
  parameters:
    cardslot: "base"
```

4.2 2. Operation: ASSERT (Validierung via XPath)

Überprüft bestimmte Werte aus dem SOAP-Antwort-Objekt. Jede fehlgeschlagene Assertion bricht den Testlauf sofort ab (Fail-Fast).

Quelle (source)	Aktion (action)	Beschreibung / Verwendung
STATUS	EQUALS , LESS_THAN , GREATER_THAN , EXITCODE	Validiert numerisch den HTTP-Statuscode (z. B. <code>expected: 200</code>).
BODY	EQUALS , NOT_EQUALS , CONTAINS	Prüft den via XPath extrahierten Textwert aus dem XML.
BODY	IS_EMPTY / IS_NOT_EMPTY	Überprüft, ob das XML-Element leer ist oder Inhalt besitzt.
HEADERS	IS_HEADER_PRESENT	Prüft, ob ein bestimmter HTTP-Header in der Antwort existiert.

YAML


```
# Inhaltsprüfung via XPath
- type: SOAP
  op: ASSERT
  response: response_soap1
  source: BODY
  path: //RemoveCardResponse/resultCode
  action: EQUALS
  expected: "OK"
```

4.3 3. Operation: BUFFER (Datenextraktion via XPath)

Extrahiert Daten mittels XPath aus dem XML-Body (oder Status/Headers) und speichert sie als globale TestCase-Variable unter dem Attribut `name` .

YAML

```
# Extrahiert eine Card-ID aus dem SOAP-Response-XML und puffert sie in "${cardid}"
- type: SOAP
  op: BUFFER
  response: response_soap1
  source: BODY
  path: //RemoveCardResponse/cardId
  name: cardid
```

5 Konsole-Logging & Live-Ausgabe (Log Alignment)

Das Logging-System stellt strukturierte und automatisch einheitlich eingerückte (formatierte) XML-Informationen bereit:

Plaintext

```
>>> CALL      : POST http://192.168.111.151/simulator/remove
>>> HEADER    : Content-Type: text/xml;charset=UTF-8
>>> HEADER    : SOAPAction: http://service.egov.at/card/RemoveCard
>>> BODY      : <soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/
envelope/">
>>> BODY      :     <soapenv:Body>
>>> BODY      :         <RemoveCardRequest>
>>> BODY      :             <slot>base</slot>
>>> BODY      :         </RemoveCardRequest>
>>> BODY      :     </soapenv:Body>
>>> BODY      : </soapenv:Envelope>
<<< HTTP      : 200
<<< HEADER    : Content-Type: text/xml;charset=utf-8
<<< BODY      : <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/
envelope/">
<<< BODY      :     <soap:Body>
<<< BODY      :         <RemoveCardResponse>
<<< BODY      :             <resultCode>OK</resultCode>
<<< BODY      :         </RemoveCardResponse>
<<< BODY      :     </soap:Body>
<<< BODY      : </soap:Envelope>

>>> SOAP ASSERT EQUALS
>>> RESULT      → SUCCESS Expected: OK, Actual: OK
```

6 Intelligentes XML-Tag-Handling: {NULL} und {EMPTY}

Im Gegensatz zu JSON können XML-Tags nicht einfach weggelassen oder als `null` definiert werden, ohne die XML-Struktur zu verändern. Die SOAP-Engine besitzt daher einen **DOM-basierten XML-Pre-Processor**, der vor dem Absenden des Requests folgende Regeln anwendet:

1. **{NULL} (Knoten-Entfernung):** Wenn der aufgelöste Textinhalt eines XML-Elements exakt "{NULL}" lautet, wird dieses Element **vollständig aus dem DOM-Baum gelöscht** (inklusive seiner schließenden Tags). Pre-Editor-Leerzeilen werden automatisch bereinigt.
2. **{EMPTY} (Leeres Tag):** Wenn der Inhalt exakt "{EMPTY}" lautet, wird der Text gelöscht, sodass ein leeres XML-Tag (`<tag></tag>` oder `<tag/>`) entsteht.

Beispiel vor der Engine-Verarbeitung:

XML

```
<Request>
  <optionalField>{E[missing_variable]}</optionalField>
<emptyField>{B[empty_variable]}</emptyField>      </Request>
```

Tatsächlich gesendetes XML (nach der Verarbeitung durch den DOM-Parser):

XML

```
<Request>
  <emptyField></emptyField>
</Request>
```